Universidad de los Andes > FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

ARQUITECTURA DE COMPUTADORES

Proyecto N°3

Grupo: 5

Integrantes:

Juan Pablo Berríos

Agustin Flores

Ana Lobo

Profesor:

Jorge Gómez

Santiago, 4 de noviembre de 2026

ÍNDICE

1. INTRODUCCIÓN 1
2. DEL RTL DE FPGA AL RTL DE ASIC 2
    2.1 Funcionamiento del Juego 2
3. MEMORIA Y PROGRAMA DEL JUEGO 5
    3.1 Estrategia 1: ROM combinacional y RAM de datos 5
    3.2 Estrategia 2: memoria unificada con flip-flops 6
    3.3 Comparación de estrategias 6
4. SÍNTESIS 8
    4.1. Corrida final y método de trazabilidad 8
    4.2. Área por módulo 9
    4.3. Celdas más utilizadas 11
        4.3.1 Composición de los flip-flops 12
    4.4 Resultado esperado versus obtenido 12
5. TIMING 15
    5.1 Corners y criterio de cierre 15
    5.2 Pruebas del período de reloj 16
        5.2.1 Reoptimización del circuito 17
    5.3 Ruta crítica 17
    5.4 Propuesta de cambio en el RTL 18
    5.5 Constantes de game.s para T = 100 ns 20
    5.6 Violaciones de slew y capacitancia 21
6. FLOORPLAN, RED DE ALIMENTACIÓN Y PLACEMENT 22
    6.1 Ubicación de los terminales 23
    6.2 Estimación de costo 24
7. SIGNOFF Y LAYOUT 25
    7.1 Violación de antena encontrada durante el desarrollo 25
    7.2 Layout anotado 26
8. SIMULACIÓN FUNCIONAL POST-SÍNTESIS 27
9. RESULTADOS Y CONCLUSIÓN 30
10. ANEXO 32
    1.1. Código: rtl/pochoco_cycle_counter.v 32
    1.2. Código: asic/config_romram.json 32
    1.3. Comandos para reproducir las corridas 33

1. INTRODUCCIÓN

En el Proyecto 2 se implementó un juego de reflejos sobre el Pochoco SoC y el procesador Espino Core, ejecutándose en una FPGA Lattice iCE40 HX1K. En una FPGA el hardware ya está fabricado: la síntesis solo configura bloques lógicos (LUTs), memorias (BRAM) y conexiones que existen de antemano. Un circuito integrado de aplicación específica (ASIC), en cambio, se construye a partir de celdas estándar que se ubican y conectan físicamente sobre el silicio, y su resultado final es un layout geométrico listo para enviarse a fabricar.

El presente proyecto tiene como objetivo llevar ese mismo SoC, con el juego del Proyecto 2, desde su descripción RTL hasta un archivo GDSII, utilizando el flujo abierto LibreLane y el PDK GF180MCU de GlobalFoundries (variante gf180mcuD, biblioteca de celdas estándar gf180mcu_fd_sc_mcu7t5v0). El resultado es un macro digital implementado con celdas estándar y no un chip completo: sus puertos son conexiones lógicas que, en un chip real, se conectarán a pads de I/O con sus circuitos de protección y manejo de carga.

Para lograrlo se adaptó el RTL de FPGA a una versión apta para ASIC, se implementaron y compararon dos estrategias de memoria para el programa del juego, y se analizaron los resultados de síntesis, timing, floorplan, signoff y simulación post-síntesis. La corrida final utiliza un período de reloj de 100 ns (10 MHz) y una utilización de core de 40 %, y se considera cerrada cuando el flujo termina con DRC y LVS limpios, sin violaciones de antena sin justificar, y con WNS ≥ 0 y TNS = 0 en setup y hold para todos los corners exigidos.

Cada decisión se justifica con datos de las propias corridas de LibreLane. Así, el informe muestra no solo el resultado final, sino también cómo cambian el área, la frecuencia máxima y las violaciones del diseño al modificar el RTL o los parámetros del flujo.

2. DEL RTL DE FPGA AL RTL DE ASIC

La migración desde la implementación FPGA del Proyecto 2 a una ASIC, consistió en separar la lógica funcional del SoC de los elementos específicos de la placa. En la versión de la FPGA, el módulo *game_top.v* generaba un reset de encendido, conectaba los periféricos y adapta las señales de los displays a la polaridad requerida por la placa. En ASIC, estas funciones se reorganizaron para que el diseño pudiera sintetizarse como un circuito independiente, así que se incorporó el módulo *asic_top.v* que constituye el módulo superior del diseño.

También se incorpora el parámetro *MemMode*, cuyo valor permite seleccionar la estrategia de memoria utilizada por el SoC. El valor *MemMode = 1* selecciona la implementación ROM RAM, mientras que *MemMode = 2* selecciona la memoria unificada implementada mediante flip-flops.

Además, se eliminó la dependencia del reset de encendido generado internamente por la FPGA. En su lugar, el ASIC recibe el reset mediante *i_Reset_n*, que se conecta al reset del SoC. También se separó la polaridad de los displays: el SoC entrega señales lógicas activas en alto, mientras que la inversión necesaria para la Go Board se mantiene únicamente en el wrapper FPGA.

2.1 Funcionamiento del Juego

Durante la migración se identificaron distintas construcciones del RTL original que dependían de características de la FPGA o de la placa utilizada. Algunas fueron detectadas durante la síntesis y otras presentan problemas de portabilidad que no producían un error en LibreLane.

Tabla 2.1: Construcciones de la FPGA y adaptaciones realizadas para ASIC.

Construcción original	Problema o evidencia	Adaptación
Inicialización de memoria mediante initial y $readnenh en pococo_ram.v	En la FPGA, el flujo de configuración puede utilizar el archivo .hex para inicializar la memoria. EN un ASIC no existe esto	Se reemplazó la carga desde archivo por generación de RTL en assembler.py. El contenido quedó en game_rom.v y game_mem_ff.v
Inferencia de memoria RAM de doble puerto en pochoco_ram.v, mediante reg [31:0]	La FPGA llegó a utilizar 12 bloques SB_RAM40_4K en el diseño. Estas primitivas no están disponibles en el flujo ASIC.	Se implementaron dos alternativas, ROM RAM, y memoria unificada construida con flip-flops
Primitivas SB_RAM40_4K en el register file original del procesador (espino_regbank32)	La jerarquía de Yosys falló al encontrar una instancia de esta primitiva. El error se produjo al intentar procesar una celda específica de la implementación FPG	Se reemplazó el register file por un arreglo genérico reg [31:0] regs [0:15], implementado con lógica secuencial estándar
Arreglo de registros genérico en espino_register_file.v	Yosys emitió un warning indicando que el arreglo se expande a registros individuales, en vez de una memoria física	Se mantuvo la implementación genérica, ya que permite mapear los registros a celdas secuenciales estándar del PDK GF180MCU
Reset de encendido FPGA en game_top.v, mediante los registros por_cnt y rst_q, inicializados en sus declaraciones	La FPGA puede establecer valores iniciales en sus registros durante la configuración. En un ASIC no se debe depender de valores iniciales implícitos de los flip-flops	Se eliminó este mecanismo del wrapper ASIC y se incorporó la entrada externa i_Reset_n. El contador de reset de encendido se conserva únicamente en el wrapper destinado a FPGA
Constantes de tiempo del programa asociadas al reloj de la FPGA	El juego medía intervalos utilizando el contador de ciclos. Y las constantes de espera dependían directamente de la frecuencia del reloj. La versión original utilizaba 25 MHz	Se ajustaron las constantes de game.s para la frecuencia objetivo de 10 MHz, es decir, 30.000.000 ciclos para tres segundos y 1.000.000 de ciclos para una décima de segundo
Inversión de las señales de los displays en game_top.v	La inversión responde a la polaridad activa en bajo de los displays de la Go Board, no a una necesidad lógica del juego.	Se dejaron las señales lógicas activas en alto en pochoco_soc.v. La inversión se conserva únicamente en el wrapper FPGA.
Señales SPI no conectadas en la implementación FPGA inicial.	En una síntesis FPGA anterior se observaron warnings de señales SPI sin driver, correspondientes a spi_cs_n_i, spi_mosi_i y spi_sclk_i	En el wrapper FPGA las señales SPI se mantienen en valores inactivos. En asic_top.v, en cambio, se exponen como terminales de entrada y salida del macro

Fuente: Elaboración propia.

En la Tabla 2.1 se presentan los cambios que permiten conservar el funcionamiento lógico del juego y del procesador, eliminando las dependencias de primitivas y mecanismos de inicialización propios de la FPGA.

3. MEMORIA Y PROGRAMA DEL JUEGO

En la implementación FPGA, el programa se almacenaba en el archivo sw/game.hex, que era leído mediante $readmemh durante la inicialización de la memoria. Este mecanismo es compatible con el flujo de configuración de una FPGA, ya que permite incorporar el contenido inicial de las memorias al configurar el dispositivo.

En un ASIC fabricado no existe un archivo .hex externo que pueda ser leído al encender el circuito. El archivo es un recurso utilizado durante el desarrollo y la síntesis, pero no constituye por sí mismo un componente físico del chip. Por esta razón, las instrucciones deben quedar representadas directamente en el hardware, ya sea mediante lógica combinacional que implemente una ROM o mediante registros que almacenen el programa.

Para estudiar el impacto de ambas alternativas se implementaron dos estrategias seleccionables mediante el parámetro MemMode de pochoco_soc.v.

3.1 Estrategia 1: ROM combinacional y RAM de datos

La primera estrategia separa la memoria de instrucciones de la memoria de datos. Para esto se modificó assembler.py, de manera que, además de generar el archivo hexadecimal tradicional, pueda generar directamente un módulo Verilog que contiene las instrucciones del programa. Primero, el assembler realiza dos pasadas, identifica las etiquetas y las direcciones de las instrucciones, y luego traduce cada instrucción a su representación binaria de 32 bits. A partir de esta lista de palabras genera rtl/game_rom.v cuando se utiliza la opción.

El módulo game_rom.v implementa una ROM combinacional mediante una sentencia case, que selecciona la instrucción correspondiente según la dirección recibida. Para este programa se generaron 97 entradas, correspondientes a las 97 instrucciones del juego. Si se consulta una dirección fuera del rango, se entrega una instrucción NOP como valor por defecto.

La lectura de instrucciones se realiza mediante el módulo pochoco_mem_romram.v. Aunque la ROM es combinacional, se registra la dirección de instrucción en instr_addr_q para conservar la latencia de un ciclo que tenía la memoria síncrona original. De esta manera, se mantiene el comportamiento temporal esperado por el procesador.

La memoria de datos se redujo a una única palabra de 32 bits, implementada mediante el registro data_word. Esta memoria conserva la interfaz de lectura y escritura del procesador, incluyendo las señales de habilitación por byte. Sin embargo, el programa del juego no utiliza memoria RAM de datos convencional: todos sus accesos de lectura y escritura corresponden a registros de periféricos mapeados en memoria (MMIO). Por lo tanto, la palabra de datos implementada se mantiene principalmente para conservar una interfaz válida con el procesador.

3.2 Estrategia 2: memoria unificada con flip-flops

La segunda estrategia implementa una memoria unificada de instrucciones y datos utilizando flip-flops, sin depender de bloques de memoria específicos de una FPGA. Al igual que en la estrategia anterior, el contenido se genera automáticamente desde sw/game.s mediante assembler.py, pero utilizando la opción --ff-mem. El resultado es el módulo rtl/game_mem_ff.v.

La memoria se dimensionó según la cantidad real de instrucciones del juego. Como el programa contiene 97 instrucciones de 32 bits, se definió una memoria de 97 palabras:

- Cantidad de palabras: 97.
- Ancho de cada palabra: 32 bits.
- Capacidad total: 3.104 bits, equivalentes a 388 bytes.
-

En el módulo generado se utiliza el arreglo reg [31:0] mem [0:96], y el parámetro NUM_WORDS se establece en 97. Durante el reset se cargan explícitamente las 97 palabras correspondientes al programa, mediante asignaciones secuenciales dentro del bloque always. Así, el contenido de la memoria queda incorporado en el RTL y no depende de la lectura de un archivo externo durante el encendido.

La memoria conserva dos interfaces: una para la lectura de instrucciones y otra para los accesos de datos. Ambas utilizan lecturas síncronas, y la interfaz de datos mantiene la posibilidad de escritura mediante habilitación por byte. Esta alternativa permite implementar la memoria utilizando celdas secuenciales estándar, aunque implica un mayor costo de área al representar el almacenamiento mediante flip-flops.

3.3 Comparación de estrategias

Tabla 3.1 Comparación de estrategias

Métrica	ROM + RAM	Flip-flops
Área de celdas (μm²)	178562	835328
Número de flip-flops	842	3976
Utilización (%)	54,35	58,3

Fuente: Elaboración propia.

En la Tabla 3.1 se presentan las dos estrategias sintetizadas y se llevó a cabo el flujo físico con LibreLane utilizando el PDK GF180MCU, con un período de reloj de 100 ns, equivalente a una frecuencia objetivo de 10 MHz.

Finalmente, se seleccionó la estrategia ROM combinacional y RAM mínima (MemMode=1) para la corrida final. Esta alternativa presentó un área de celdas estándar de 178.562 μm², frente a 835.328 μm² de la implementación con flip-flops. Esto representa una reducción aproximada del 78,6 % en el área de celdas estándar. También requirió menos celdas secuenciales: 842 frente a 3.976.
Ambas implementaciones completaron el flujo físico y superaron las verificaciones de DRC, LVS y antena, sin violaciones reportadas en TNS de setup y hold para el período de 100 ns. Por lo tanto, la elección de ROM + RAM no se debe a que la alternativa con flip-flops haya fallado funcionalmente, sino a que permite obtener una implementación considerablemente más pequeña, manteniendo el comportamiento requerido por el juego a la frecuencia objetivo.

4. SÍNTESIS

4.1. Corrida final y método de trazabilidad

La corrida final utiliza la estrategia de ROM combinacional (*config_romram.json*, parámetro MemMode = 1) con un período de 100 ns y una utilización de core de 40 %. La síntesis la realiza Yosys dentro de LibreLane; los datos de esta sección provienen del reporte *stat.rpt* del paso de síntesis, antes de placement y routing.

Herramienta o recurso	Versión utilizada
LibreLane	3.0.17
PDK	gf180mcuD (open_pdks 54435919abffb937387ec956209f9cf5fd2dfbee)
Biblioteca de celdas estándar	gf180mcu_fd_sc_mcu7t5v0

Por defecto LibreLane aplana toda la jerarquía durante la síntesis: el reporte muestra un único bloque y las celdas pierden el nombre del módulo al que pertenecían (aparecen como *_3699_*, *_4204_*, etc.). Como el enunciado exige identificar al menos el register file, la ALU y el contador de ciclos, se usó la variable **SYNTH KEEP HIERARCHY MODULES**, que aplana todo el diseño excepto los módulos indicados:

```
"SYNTH_KEEP_HIERARCHY_MODULES": [
"espino_register_file",
"espino_alu",
"pochoco_cycle_counter"
]
```

En el Proyecto 2 el contador de ciclos era un registro dentro de *pochoco_periph.v*, por lo que no tenía un módulo propio que conservar. Se movió a un módulo nuevo, *pochoco_cycle_counter.v*m, implementado con un reset asíncrono, un incremento con límite de 32 bits e instanciado como *u_cycle_counter* dentro de los periféricos. Con esto, las tres partes quedan identificadas por sus nombres jerárquicos:

- Register file: *u_soc.u_core.**u_rf***
- ALU: *u_soc.u_core.u_id.**u_alu***
- Contador de ciclos: *u_soc.u_periph.**u_cycle_counter***

Se utilizó un comando para obtener información sobre los módulos que no fueron aplanados ni optimizados por la síntesis, conservándolos desde el diseño original. Estos módulos fueron 4, siendo 3 de estos los anteriormente mencionados.

```
module asic_top(i_Clk, i_Reset_n, o_LED, i_Switch, o_Segment1_A,
o_Segment1_B, o_Segment1_C, o_Segment1_D, o_Segment1_E, o_Segment1_F,
o_Segment1_G, o_Segment2_A, o_Segment2_B, o_Segment2_C, o_Segment2_D,
o_Segment2_E, o_Segment2_F, o_Segment2_G, i_SPI_SCLK, i_SPI_MOSI,
i_SPI_CS_n

module espino_alu(operator_i, operand_a_i, operand_b_i, result_o,
cmp_result_o);

module espino_register_file(clk_i, rst_ni, raddr_a_i, rdata_a_o,
raddr_b_i, rdata_b_o, waddr_i, wdata_i, we_i);

module pochoco_cycle_counter(clk_i, rst_ni, count_o);
```

Conservar la jerarquía tiene un costo, porque la síntesis ya no puede optimizar la lógica que cruza los bordes de esos módulos. Comparando la misma corrida a 100 ns con y sin jerarquía, el área final de celdas pasó de 172.642 µm² a 174.457 µm² (+1,05 %) y los buffers de reparación de timing de 688 a 739. El número de flip-flops no cambió (842).

4.2. Área por módulo

La sección *design hierarchy* del reporte de síntesis *stat.rpt* entrega el área de cada módulo conservado. Todo lo que se aplanó queda contabilizado en el módulo superior *asic_top*:

Módulo (instancia)	Celdas	Área (µm²)	% del total	Área secuencial (µm²)
espino_register_file	1.804	71.603	53,5 %	40.602 (56,7 %)
espino_alu	580	10.006	7,5 %	0
pochoco_cycle_counter	114	3.943	2,94 %	2.388 (60,6 %)
asic_top	2.077	48.336	36,1 %	18.758 (38,8 %)
Total	**4.575**	**133.887**	**100 %**	**61.749 (46,1 %)**

```
=== asic_top ===
   Chip area for module '\asic_top': 48336.108800
     of which used for sequential elements: 18757.984000 (38.81%)
=== espino_alu ===
   Chip area for module '\espino_alu': 10005.721600
     of which used for sequential elements: 0.000000 (0.00%)
=== espino_register_file ===
   Chip area for module '\espino_register_file': 71603.033600
     of which used for sequential elements: 40602.419200 (56.70%)
=== pochoco_cycle_counter ===
   Chip area for module '\pochoco_cycle_counter': 3942.579200
     of which used for sequential elements: 2388.377600 (60.58%)
=== design hierarchy ===
   Chip area for top module '\asic_top': 133887.443200


=== design hierarchy ===

        +----------Count including submodules.
        |        +-Area including submodules.
        |        |
     4575 1.34E+05 asic_top
      580    1E+04 espino_alu
     1804 7.16E+04 espino_register_file
      114 3.94E+03 pochoco_cycle_counter

        +----------Count including submodules.
        |        +-Area including submodules.
        |        |
     4595        - wires
     4830        - wire bits
     1013        - public wires
     1248        - public wire bits
       39        - ports
      274        - port bits
        -        - memories
        -        - memory bits
        -        - processes
     4575 1.34E+05 cells
     . . .
```

El módulo que aporta más área es el **“register_file”**, con un 53,5 % del total: ocupa más que todo el resto del procesador y del SoC juntos. Esto se explica porque sus 15 registros de 32 bits están construidos con flip-flops y uniéndose a registros de salida, donde cada uno de esos bits necesita además un multiplexor de escritura y una entrada en los multiplexores de lectura.

El **“cycle_counter”** agregado en el Proyecto 2 ocupa 3.943 µm², un 2,94 % del área de síntesis. De ese valor, 2.388 µm² (60,6 %) corresponden a sus 32 flip-flops y el resto al incrementador de 32 bits.

En total las celdas secuenciales forman el 46,1% del área de síntesis.

4.3. Celdas más utilizadas

Las cinco celdas más usadas del diseño completo, según la sección *design hierarchy* de *stat.rpt*, son:

#	Celda	Cantidad	Parte del diseño que la explica
1	dffrnq_1 (FF con reset)	738	Hay 544 en el register file (15 × 32 bits de registros + 2 × 32 bits de salidas registradas) 32 en el contador 162 en el resto
2	nand2_1	669	143 en el register file 82 en la ALU 6 en el contador 438 en el resto
3	aoi22_1	543	384 en el register file (debido a sus multiplexores de lectura) 9 en la ALU 150 en el resto
4	mux2_2	540	480 en el register file (15 registros × 32 bits: cada bit elige entre mantener su valor o escribir el dato nuevo) 60 en el resto
5	nor2_1	280	12 en el register file 80 en la ALU 8 en el contador 180 en el resto

La cantidad de *aoi22_1* se explica por cómo Yosys construyó la lectura del register file. Cada bit de cada uno de los dos puertos de lectura es un multiplexor de 15 entradas (el registro x0 no se almacena), armado con 6 celdas *aoi22* (12 entradas), 1 celda *aoi222* (3 entradas) y 2 celdas *and4*. Para 32 bits y 2 puertos resultan 6 × 32 × 2 = 384 celdas *aoi22*, 64 celdas *aoi222* y 128 celdas *and4*, valores que coinciden exactamente con el reporte.

4.3.1 Composición de los flip-flops

El diseño tiene 842 flip-flops: 738 *dffrnq* (reset asíncrono a 0), 101 *dffq* (sin reset) y 3 *dffsnq* (reset asíncrono negado, es decir, reset a 1). Los 3 *dffsnq* corresponden al sincronizador *cs_sync* del SPI, que se inicializa en 3'b111.

4.4 Resultado esperado versus obtenido

Antes de revisar el reporte se estimó el área de utilización contando el espacio de almacenamiento y las compuertas lógicas de cada módulo en el RTL, utilizando las áreas de las celdas propuestas por el PDK y multiplicándolas con los registros declarados en cada módulo.

Aspecto	Esperado	Obtenido	Cálculo
FFs totales	895	842	Suma de todos los FFs
FFs del register file	576	544	**16 x 32 + 2 x 32** regs[0:15]: 16 × 32 rdata_a_o y rdata_b_o: 2 × 32
FFs de la ALU	0	0	Es combinacional, no utiliza FFs
FFs del contador	32	32	count_o: 1 × 32
FFs de instr_addr_q	32	7	reg [31:0] instr_addr_q
FFs en la salida de la ROM	0	30	La ROM es un case combinacional, por lo que no utiliza FFs
FFs de rdata_o del SPI	32	8	output reg [31:0] rdata_o
FFs de mosi_sync y rx_shift (SPI)	11	9	reg [2:0] mosi_sync + reg [7:0] rx_shift

Aspecto	Esperado	Obtenido	Cálculo
Área del register file	≈ 91.900 µm²	71.603 µm²	576 FF × 74,64 + 512 mux2 de escritura × 32,93 + 960 mux2 de lectura × 32,93 (2 puertos × 32 bits × 15 mux2) + ≈ 400 del decodificador
Área de la ALU	≈ 16.600 µm²	10.006 µm²	Sumador y restador de 32 full adders cada uno + comparadores de igualdad y de magnitud (con y sin signo) + 32 × (XOR, OR, AND) + mux de resultado de 128 mux2
Área del contador	≈ 3.750 µm²	3.943 µm²	32 FF × 74,64 + 32 half adders × 43
Módulo más grande	Register file	Register file	Solo sus FFs (≈ 40.600 µm²) superan cualquier otro bloque estimado

Las diferencias que se pueden visualizar entre lo esperado y lo obtenido en cuanto al número de Flip Flops, se da en gran parte a las decisiones de optimización de Yosys y la escritura de código Verilog con variables no utilizadas.

Por ejemplo, para los casos en que se esperaban mas de los obtenidos, se debía en que algunos de los bits de la declaracion completa no se usaban

- *instr_addr_q*: los bits addr_i[31:9] y [1:0] no se usan, por eso *instr_addr_q* queda en 7 flip-flops, envés del total 32 definido.
- *register_file:* Para el register file nunca se utiliza el registro x0 para escribir, por lo que se elimina, quitando 32 Flip Flops de la lógica
- *SPI:* Sucede lo mismo con módulos en el SPI, donde se declaran más bits de los que se utiliza en la práctica

También existe una diferencia en el ROM, que fue programado como un módulo combinacional usando “case” para las diferentes opciones, debido a esto, Yosys implementó Flip Flops para guardar el valor del case, usando un multiplexor de decisión entre *stall* o continuar con las instrucciones. Se guardan 30 bits en vez de 32 porque los primeros bits de una instrucciones en RV32 siempre parten con 11, por lo que se optimizó de esa manera.

Otras diferencias se ven en el área utilizada por los distintos módulos. Para el **register file** es simplemente una decisión de construcción, donde Yosys reemplazo los mux utilizados por compuertas *aoi*, ademas de la menor cantidad de FFs vista enteriormente.

El cambio en área mayor se da en el módulo ALU, ya que el cálculo hecho asume que cada circuito es independiente, necesitando un operador, sumador, restador, etc.… para cada circuito, mientras que Yosys decidió hacer estos módulos compartidos entre todos, ya que al realizar una operación por ciclo nunca se utiliza en simultáneo con otra operación.

Por último, el módulo más grande fue predicho exitosamente, ya que el área que utilizarían teóricamente sus Flip Flops era mayor a cualquier módulo (40.6 um^2 > 16.6 um^2 ALU). Cabe destacar que el área de síntesis (133.887 μm^2) no es el área final del diseño. En la corrida final el área de celdas es 174.448 μm^2, diferencia vista al agregar el resto del flujo con 739 buffers de reparación de timing, árbol de reloj, entre otros…

5. TIMING

5.1 Corners y criterio de cierre

El análisis estático de timing (STA) de LibreLane se ejecuta en 9 corners, que combinan tres condiciones de proceso, voltaje y temperatura, definiendo la velocidad de los transistores con tres estimaciones de la resistencia y capacitancia de los cables (nom, min y max). El nombre de cada córner sigue el formato <extracción>_<proceso>_<temperatura>_<voltaje>:

Condición	Proceso	Temperatura	Voltaje	Peor caso para
tt	Típico	25°C	5,0 V	Ninguno
ss	Lento	125°C	4,5 V	Setup
ff	Rápido	- 40°C	5,5 V	Hold

Los nueve corners vistos en consola son:

```
max_ff_n40C_5v50
max_ss_125C_4v50
max_tt_025C_5v00
min_ff_n40C_5v50
min_ss_125C_4v50
min_tt_025C_5v00
nom_ff_n40C_5v50
nom_ss_125C_4v50
nom_tt_025C_5v00
```

Por defecto LibreLane solo considera error una violación en los corners típicos. Para que el flujo falle si cualquiera de los 9 corners viola setup o hold, se agregó al config:

```
"TIMING_VIOLATION_CORNERS": ["*"]
```

Una corrida se considera cerrada cuando cumple WNS ≥ 0 y TNS = 0 en setup y hold en todos los corners, además de DRC, LVS y pruebas de antenas.

5.2 Pruebas del período de reloj

Se barrió CLOCK_PERIOD en cinco puntos usando la misma configuración de la corrida final. La tabla muestra el peor slack (WNS) y el slack negativo total (TNS) de setup y hold en el corner típico (nom_tt_025C_5v00) y en el más lento (max_ss_125C_4v50), tomados de *summary.rpt* del paso de STA post-PnR. Los valores están en nanosegundos.

Setup

Clock Period	WNS típico	TNS típico	WNS lento	TNS Lento	¿Cierra?
100	+68,285	0	+39,659	0	Sí
65	+33,285	0	+4,659	0	Sí
61	+29,285	0	+0,659	0	Sí
60,5	**+28,785**	**0**	**+0,159**	**0**	**Sí**
60,3	+28,585	0	−0,041	−0,101	No (7 endpoints)

Hold

Clock Period	WNS típico	TNS típico	WNS lento	TNS Lento	¿Cierra?
100	+0,431	0	+1,050	0	Sí
65	+0,431	0	+1,050	0	Sí
61	+0,431	0	+1,050	0	Sí
60,5	**+0,431**	**0**	**+1,050**	**0**	**Sí**
60,3	+0,431	0	+1,050	0	No (7 endpoints)

El menor período que cierra timing en todos los corners es **60,5 ns**, lo que corresponde a una frecuencia máxima de **16,5 MHz** (1000/60.5). Todas las corridas que cierran pasan también DRC, LVS y antena. A 60,3 ns el diseño falla solo en el corner max_ss_125C_4v50, por 0,041 ns (TNS de −0,101 ns), mientras los otros 8 corners cumplen. Por lo tanto, el límite real está entre 60,3 y 60,5 ns. . En todo el barrido el hold se cumple con holgura y su TNS es 0, porque el período no afecta las restricciones de hold.

5.2.1 Reoptimización del circuito

Cada valor de CLOCK_PERIOD puede provocar una nueva optimización del circuito y, por lo tanto, modificar el área, el buffering y el placement, ya que el flujo repara las rutas que no cumplen agregando o redimensionando buffers. En este barrido eso no ocurrió: los slacks de setup bajan exactamente lo mismo que el período (de 100 a 65 ns, ambos corners pierden 35,00 ns) y los slacks de hold son idénticos en todas las corridas. Es decir, el netlist, el placement y la ruta crítica son los mismos.

La razón es que la reparación de timing de LibreLane trabaja por defecto solo en el corner típico (PNR CORNERS sin definir). En ese córner sobran más de 28 ns incluso a 60,5 ns, así que el reparador no encuentra nada que corregir. La violación solo existe en el corner lento, que el reparador no mira.

5.3 Ruta crítica

La ruta crítica se obtuvo del reporte *max.rpt* del córner *max_ss_125C_4v50*. Como el circuito no se reoptimiza entre corridas, es la misma ruta en todo el barrido.

Elemento	Valor
Celda de inicio	**_3699_**: flip-flop hacia a_rdata_o[3] de la memoria, el bit 3 de la instrucción que registra la salida de la ROM
Módulo de inicio	pochoco_mem_romram
Celda de fin	**_2867_**: un flip-flop del register file (bit 27 del dato escrito)
Modulo fin	espino_register_file
Módulos que recorre	Memoria de instrucciones → decoder → ALU → load-store unit → register file
Retardo en el corner lento	≈ 59.42 ns
Función de Espino	Decode (tramo dominante, 44,9 % del retardo), en una ruta que continúa por ejecución, acceso a memoria y write-back en el mismo ciclo

Siguiendo los nombres de las redes en el reporte, el recorrido es: la instrucción (*a_rdata_o[3]*) entra a la lógica del decoder, pasa por el multiplexor que elige el segundo operando de la ALU entre el inmediato y rs2 (*operand_b*), atraviesa la ALU, cuyo resultado es la dirección del acceso a memoria (*lsu_addr*), sigue por la lógica de alineación de la LSU y llega como dato de escritura (*rf_wdata[27]*) al multiplexor de escritura del register file.

Tramo	Retardo (ns)	% de la ruta
Registro de instrucción (clk → Q)	2,06	3,5 %
Decode y mux de operando	26,68	44,9 %
Ejecución (ALU)	11,95	20,1 %
Acceso a memoria (LSU)	14,99	25,2 %
Write-back (register file)	3,75	6,3 %
Total	**59,42**	**100 %**

5.4 Propuesta de cambio en el RTL

En Espino un load tarda dos ciclos: en el primero se envía la dirección a la memoria y en el segundo llega el dato, se alinea y se escribe en el register file. Durante el segundo ciclo la dirección ya se conocía desde el primero, pero el RTL vuelve a usar la salida de la ALU para elegir el byte.

La propuesta es registrar estos dos bits cuando se emite el load y usar el registro en la alineación del segundo ciclo, de modo que la ALU salga de esa cadena. El cambio se implementó en *espino_load_store_unit.v*:

```
`ifdef LSU_REG_BOFF
  reg [1:0] boff_q;
  always @(posedge clk_i or negedge rst_ni) begin
    if (!rst_ni)                 boff_q <= 2'b00;
    else if (is_load && !busy_q) boff_q <= addr_i[1:0];
  end
  wire [1:0] load_boff = boff_q;
`else
  wire [1:0] load_boff = boff;
`endif
     wire  [15:0]  lhalf  =  load_boff[1]  ?  data_rdata_i[31:16]  :
data_rdata_i[15:0];
  wire [7:0]  lbyte = load_boff[0] ? lhalf[15:8]         : lhalf[7:0];
```

boff_q se carga solo en el primer ciclo del load (*is_load && !busy_q*) y conserva su valor en el segundo. Los stores siguen usando boff combinacional, porque escriben en el mismo ciclo. La cantidad de ciclos por instrucción no cambia, así que el programa y sus constantes siguen siendo válidos. El cambio queda condicionado a la macro *LSU_REG_BOFF*, que se activa con *VERILOG_DEFINES* en un config separado *(config_romram_lsu.json*). Así, la corrida final a 100 ns y los resultados de síntesis de la sección 4 corresponden al RTL original.

Ambas versiones se compararon con el mismo período (60,5 ns):

Métrica	RTL original	boff registrado
Setup WNS, corner lento (ns)	+0,159	+1,954
Setup WNS, corner típico (ns)	+28,785	+29,687
Peor hold de los 9 corners (ns)	+0,167	+0,132
Flip-flops	842	844
Área de síntesis (μm²)	133.887	133.690
Área final de celdas (μm²)	174.448	174.277
Buffers de reparación	739	745
Período mínimo (ns)	60,5	58,6
Frecuencia máxima (MHz)	16,53	17,06

Con este cambio, la ruta crítica es diferente

Elemento	Valor
Celda de inicio	***_3610_***, flip-flop con a_rdata_o[4] (bit 4 de la instrucción)
Celda de fin	***_3540_***, {[registro que guarda]}
Recorrido	Memoria de instrucciones → decoder → mux de operando → ALU → bit 31 de la dirección → decodificación de la dirección y habilitación de registros

Tramo	Retardo (ns)	% de la ruta
Registro de instrucción (clk → Q)	2,16	3,7 %e
Decode y mux de operando	21,65	37,5 %
ALU (bit 31 de la dirección)	14,97	25,9 %
Decodificación de la dirección y habilitación	18,97	32,8 %
Total	**57,75**	**100 %**

Como se puede, la optimización fue pequeña, mejorando el *slack* en un 5% y bajando el área de síntesis y final en pequeñas cantidades.

5.5 Constantes de game.s para T = 100 ns

En el Proyecto 2 el juego corría a 25 MHz. Si el macro funciona con un período de 100 ns, la frecuencia es f = 1 / 100 ns = 10 MHz, y deben cambiar las dos constantes que dependen del reloj: la espera de tres segundos y el divisor que convierte ciclos a décimas de segundo.

Constante	Cálculo	Decimal	Hexadecimal	Instrucciones
Espera de 3 s	3 s × 10.000.000 Hz	30.000.000	0x01C9C380	lui x9, 0x1C9C addi x9, x9, 0x380
Una décima de segundo	0,1 s × 10.000.000 Hz	1.000.000	0x000F4240	lui x9, 0xF4 addi x9, x9, 0x240

Luego de hacer los cambios en las constantes, se debe correr el comando “make asm” desde la raíz del proyecto, así el assembler genera el game.s y las instrucciones ASIC con las configuraciones de romram y ff.

5.6 Violaciones de slew y capacitancia

Aunque no forman parte de los criterios de cierre del enunciado, el flujo reporta violaciones de slew (tiempo de subida o bajada de una señal, límite de 3 ns) y de capacitancia máxima. En la corrida final se distribuyen así:

Corner	tt	ss	ff
Slew (nom / min / max)	11 / 0 / 173	1.807 / 1.440 / 2.251	0 / 0 / 0
Capacitancia (nom / min / max)	1 / 1 / 6	1 / 1 / 6	1 / 1 / 6

Las violaciones de slew se concentran en los corners lentos, donde los transistores entregan menos corriente y los cables tienen más capacidad que cargar, y no aparecen en los rápidos. Ocurren donde celdas de fuerza mínima manejan cargas grandes.

La violación de capacitancia presente en todos los corners corresponde a la raíz del árbol de reloj, que maneja los cables que cruzan todo el chip. El árbol también incumple el fanout máximo genérico de 10: cada buffer hoja maneja entre 16 y 19 flip-flops. Esto se debe a que el árbol de reloj se construye con celdas de reloj de alta fuerza (clkbuf_8 y clkbuf_16) para que el reloj llegue a todos los flip-flops al mismo tiempo, y no para cumplir el límite de fanout pensado para señales de datos

6. FLOORPLAN, RED DE ALIMENTACIÓN Y PLACEMENT

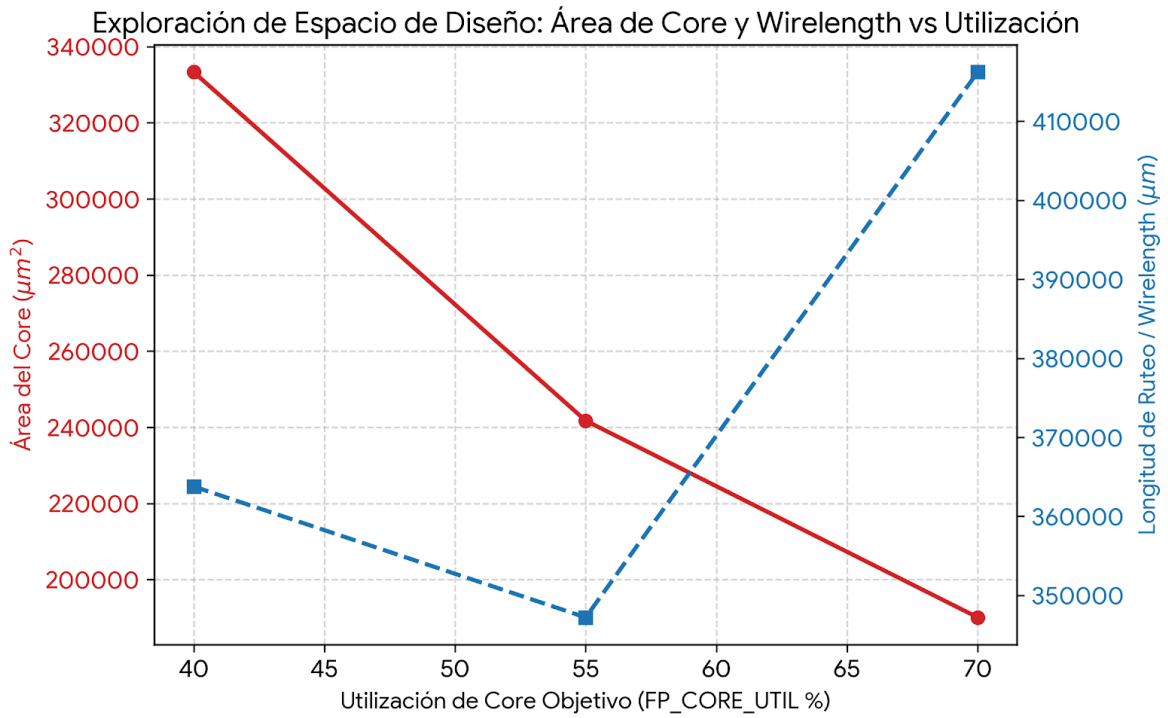
Para determinar las dimensiones óptimas y comprobar el impacto de la densidad de colocación, se ejecutó un barrido del parámetro de utilización del núcleo (*FP_CORE_UTIL*) evaluado en tres puntos: 40%, 55% y 70%.

A partir de los reportes extraídos de *LibreLane* (*metrics.csv* y logs de cierre PnR), se consolida el siguiente cuadro comparativo:

Parámetro/Métrica	**Corrida 1 (40%)**	**Corrida 2 (55%)**	**Corrida 3 (70%)**
Área del Core (μm^2)	333.343	241.746	190.060
Área del Die (μm^2)	361.061	265.959	211.281
Utilización real (%)	52,33%	70,96%	89,18%
Wirelength Total (μm)	363.778	347.208	416.236
Violaciones DRC (Magic)	0	0	0
Setup WNS Típico (ns)	+68, 28	+69, 13	+69, 01
Cierre de Timing (WNS $\geq$ 0, TNS = 0)	Si	Si	Si

Como se puede observar, los tres puntos evaluados completaron exitosamente el flujo sin presentar errores de diseño geométrico (0 errores de DRC con Magic), cumpliendo los criterios de timing estricto en todos los esquemas.

Al aumentar la densidad objetivo de core de 40% a 70%, el área total del dado se reduce drásticamente de 361.061 μm^2 a 211.281 μm^2. Sin embargo, al llegar a una utilización real del 89,18% (con *FP_CORE_UTIL* = 70%), la congestión local obliga a un ruteo más sinuoso, aumentando la longitud total del cableado (*wirelength*) a 416.263 μm en comparación con los 363.778 μm de la corrida base al 40%. La elección oficial de 40% garantiza un excelente margen de holgura para el ruteo global y la inserción de celdas de relleno sin comprometer la integridad eléctrica.



Fuente: Elaboración propia

6.1 Ubicación de los terminales

En el macro *asic_top*, los puertos de entrada y salida representan las terminales lógicas del circuito y no pads físicos de I/O aptos para soportar corrientes de carga externas o proteger contra descargas electrostáticas (ESD). Su distribución en los bordes del área de diseño se estructuró de manera funcional:

- **Señales de Sincronización y Control:** Los pines de reloj (*i_Clk*) y reset (*i_Reset_n*) se ubicaron de forma central o próxima al punto de inyección del árbol del reloj para minimizar el sesgo (*skew*) inicial.
- **Interfaz de Comunicaciones (SPI):** Las líneas de la interfaz SPI (*i_SPI_SCLK*, *i_SPI_MOSI*, *i_SPI_CS_n*, *o_SPI_MISO*) se agruparon en un mismo flanco del dado para mantener las trazas compactas y evitar acoples inductivos.
- **Periféricos de Usuario:** Los buses de control humano (*i_Switch* y *o_LED*) y las líneas multiplexadas de los displays de 7 segmentos se distribuyeron en los bordes restantes del macro en correspondencia directa con la orientación de los pines de la tarjeta de desarrollo y los bloques lógicos internos del SoC.

6.2 Estimación de costo

Tomando como referencia la corrida final con una utilización de core del 40%, el área total del dado (*DIE_AREA*) obtenido fue de aproximadamente 0,36 mm^2 (361.061 μm^2). Utilizando la métrica orientativa general del mercado de US$23.000 por mm^2:

$$Costo\ Estimado\ =\ 0,361\ mm^2\ \bullet\ US\$23.000/mm^2\ =\ US\$8.303$$

También hay que recordar las limitaciones de la aproximación lineal y los factores de impacto existentes:

- **Áreas mínimas de fundición:** Las fundiciones reales exigen áreas mínimas de bloque obligatorios (1 mm^2 para bloques comerciales o multi-proyecto). Por lo tanto, un diseño pequeño de 0,36 mm^2 igual debe sufragar el costo del bloque entero
- **Costos fijos adicionales:** El costo real no depende exclusivamente del silicio, sino de tarifas de acceso al PDK, costos de empaquetado (*packaging*), fabricación de PCBs de prueba y envíos internacionales.
- **Decisión con mayor impacto:** La decisión arquitectónica que dominó el área del macro (y por ende el costo equivalente) fue la elección de la estrategia de memoria. El uso de flip-flops y la lógica de decodificación del *register file* consumen más del 53% del área de celdas estándar, superando ampliamente el costo físico de la ALU y los periféricos.

7. SIGNOFF Y LAYOUT

Una vez realizada la corrida física del macro *asic_top* bajo el flujo de *LibreLane* con un período de reloj objetivo de $T = 100ns$ y una utilización de core de 40%, se ejecutaron las validaciones finales de *signoff* físico y eléctrico:

- **DRC (*Design Rule Check - Magic*):** La verificación geométrica con Magic reportó 0 errores de DRC (*magic__drc_error__count = 0* y *magic__illegal_overlap__count = 0*), lo que garantiza que las dimensiones, espaciamientos y anchos de las capas de metal y óxido cumplen rigurosamente con las reglas de fabricación del PDK GF180MCU.
- **LVS (*Layout Versus Schematic - Netgen*):** El reporte de comparación eléctrica confirmó que el diseño final cierra limpio: *Circuits match uniquely*. Las métricas arrojan un conteo de errores nulo (*design__lvs_error__count = 0*, dispositivos y reds desalineadas en cero). validando que el layout físico generado en el GDSII es idéntico a nivel de conectividad al netlist lógico sintetizado.
- **Verificación de Antenas:** El enrutador global incorpora la mitigación de efectos de antena mediante la inserción automática de diodos de protección (*antenna_diodes_count = 4, design__instance__count__class:antenna_cell = 5*). Gracias a esto, el reporte final confirma un estado limpio sin redes ni pines que violen los radios de antena permitidos (*antenna__violating__nets = 0* y *antenna__violating__pins = 0*).

7.1 Violación de antena encontrada durante el desarrollo

Durante las iteraciones preliminares del diseño físico (especialmente al preservar la jerarquía de los módulos como el *register file*), se detectaron alertas de acumulación de carga por plasma en trazas de alta fanout dentro de la capa Metal 2.

- **Impacto:** Estas trazas actúan como antenas durante el grabado por plasma, pudiendo perforar el dielétrico de compuerta de los transistores conectados.

- **Resolución:** Se ajustó el enrutador global en el archivo de configuración agregando un margen superior de iteraciones y holgura de ruteo (“*GRT_ANTENNA_REPAIR_ITERS” : 10, “GTR_ATENNA_REPAIR_MARGIN”: 20*) lo que permitió la inserción limpia de los diodos de antena sin comprometer el cierre del timing.

7.2 Layout anotado

En la siguiente figura se muestra el layout físico post-PnR del macro *asic_top* visualizado en OpenROAD GUI, donde se ha resaltado y segregado espacialmente los bloques clave preservados mediante la directiva de jerarquía:

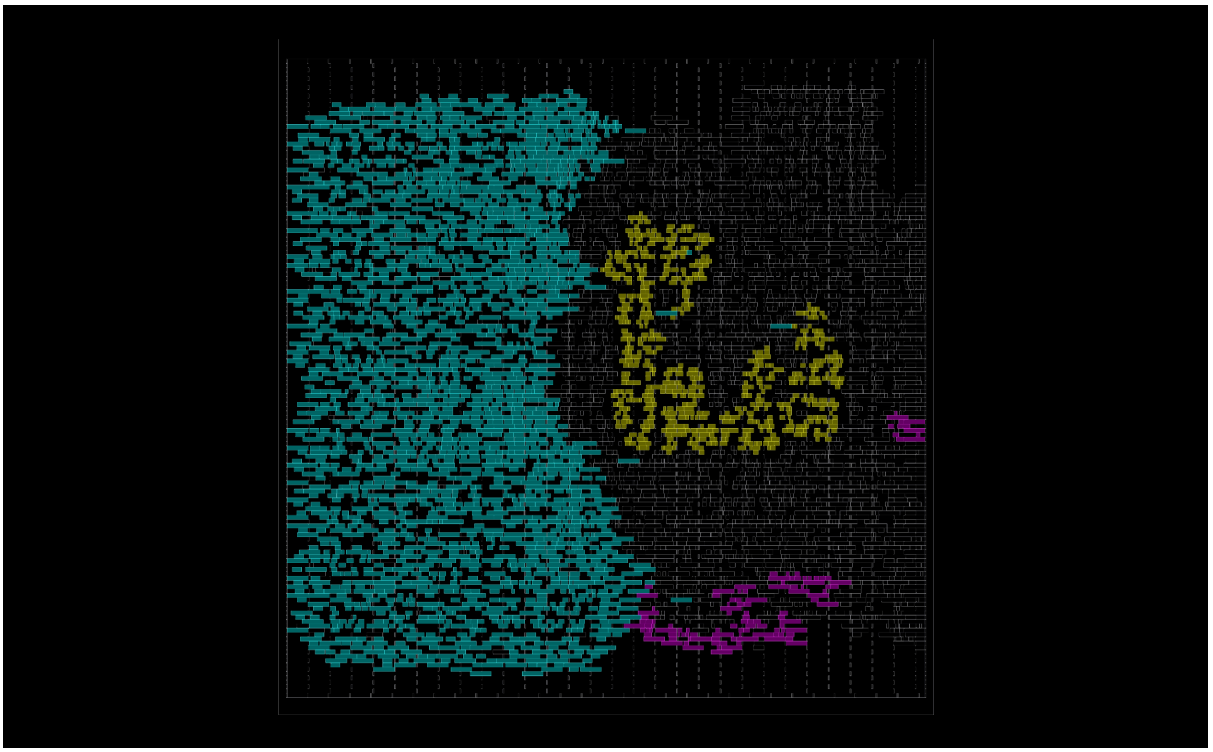


Figura 7.1: Vista superior de layout físico del macro *asic_top* destacando los bloques conservados (OpenROAD GUI).

En la imagen se puede destacar el *Register File* en color cian, la ALU en amarillo y el contador de ciclos en magenta. El layout ha adaptado esta forma por los siguientes motivos:

- El *register file* está concentrado densamente debido al elevado número de flip-flops y multiplexores que lo componen, ubicándose estratégicamente para minimizar la longitud de los buses de datos hacia la ALU.
- La ALU se sitúa contigua al bloque de registros para optimizar los tiempos de llegada en los flancos de reloj críticos .
- La red de distribución de energía (*Power Grid*) y las pistas de interconexión completan el enrutamiento global sin violar los espacios de separación geométrica, logrando un diseño 100% limpio de DRC y LVS.

8. SIMULACIÓN FUNCIONAL POST-SÍNTESIS

Para verificar el correcto funcionamiento microscópico y lógico del diseño tras el proceso de síntesis realizado por *LibraLane*, se ejecuto una simulaciones nivel de compuertas (*Gate-Level Simulation*) utilizando el netlist final si retardos anotados (*asic_top.nl.v*).

Con el fin de hacer viable la ejecución temporal de la simulación sin alterar la lógica de control del juego, se redujeron temporalmente los contadores en el código *Assembly* de prueba (*game_sim_4.s*): la espera de inicio se fijó en 100 ciclos y el divisor de tiempo para las décimas se ajustó a 10 ciclos por décima. Se aplicó exactamente la misma configuración en la simulación de referencia en RTL y en la simulación post-síntesis.

El banco de pruebas automatizado (*sim/tb_sim.v*) se encargó de inyectar el reloj a un periodo de 100 ns y presionar los botones (*i_Switch*) en ciclos exactos, validando dos escenarios posibles:

- **Prueba pulsación incorrecta:** Al inyectar una pulsación errónea en los botones de entrada durante el transcurso de una ronda activa, el sistema respondió de inmediato mostrando el código del error (EE) en el display de 7 segmentos y apagando/reiniciando los LEDs al estado de espera (*4'b1111*), confirmando que la ronda fallida se reinició correctamente sin sumar puntos al contador de aciertos.
- **Prueba de ronda correcta:** Al inyectar la pulsación correcta en el ciclo de reloj esperado, el sistema apagó los LEDs (*4'b0000*) indicando el acierto y ejecutó la rutina aritmética de restas sucesivas para calcular las décimas de segundo. Al monitoerear el registro interno de tiempo de reacción (x15 en *u_soc.u_core.u_rf.regs[15*) y las señales de salida del display, se comprobó que el número de ciclos medidos es exactamente idéntico entre la simulación RTL y la simulación post-síntesis a nivel de compuertas.

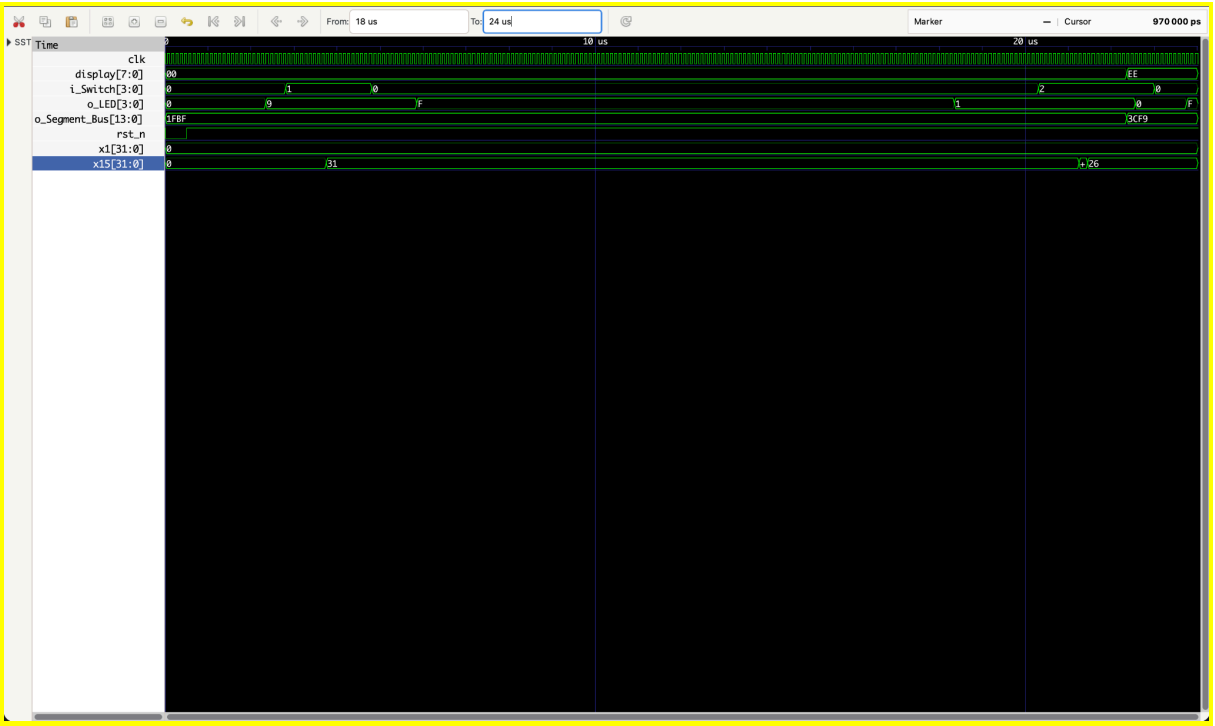


Figura 8.1: Simulación post-síntesis a nivel de compuertas mostrando el código de error y el reinicio de los LEDs ante una pulsación incorrecta.

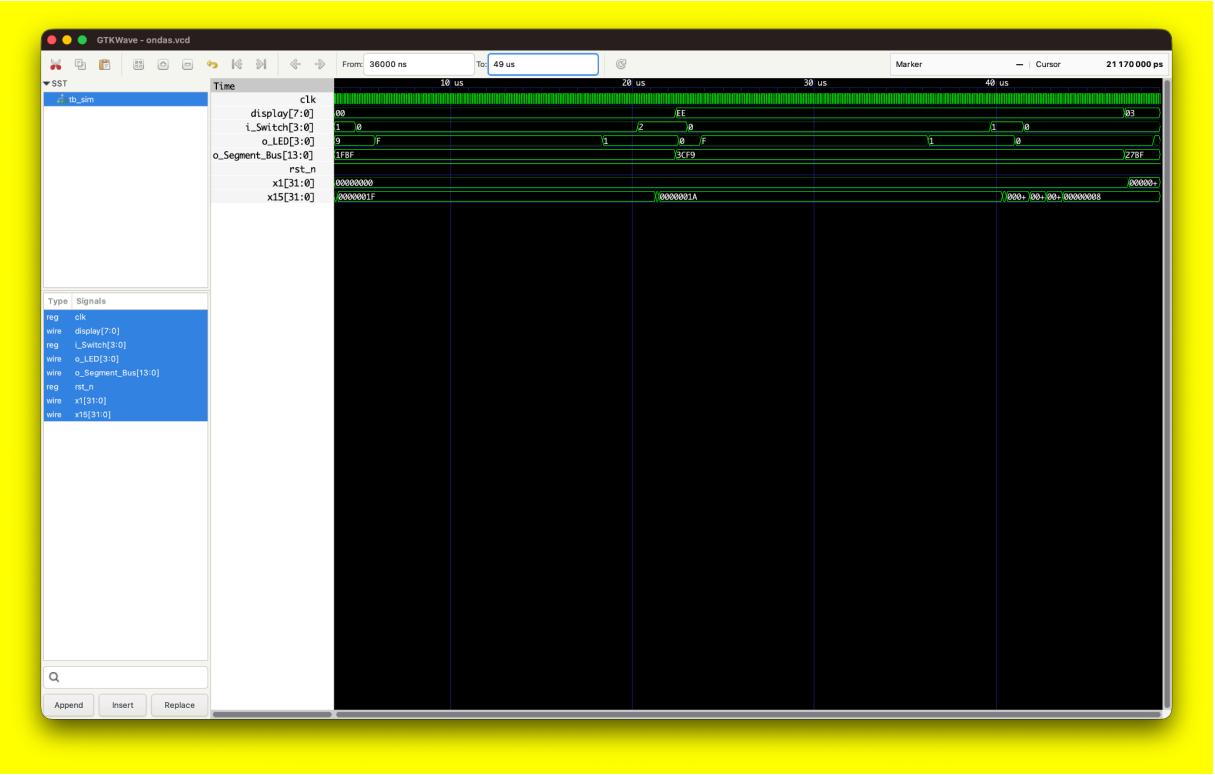


Figura 8.2: Simulación RTL de referencia para una ronda correcta, capturando el tiempo de reacción exacto en el registro x15

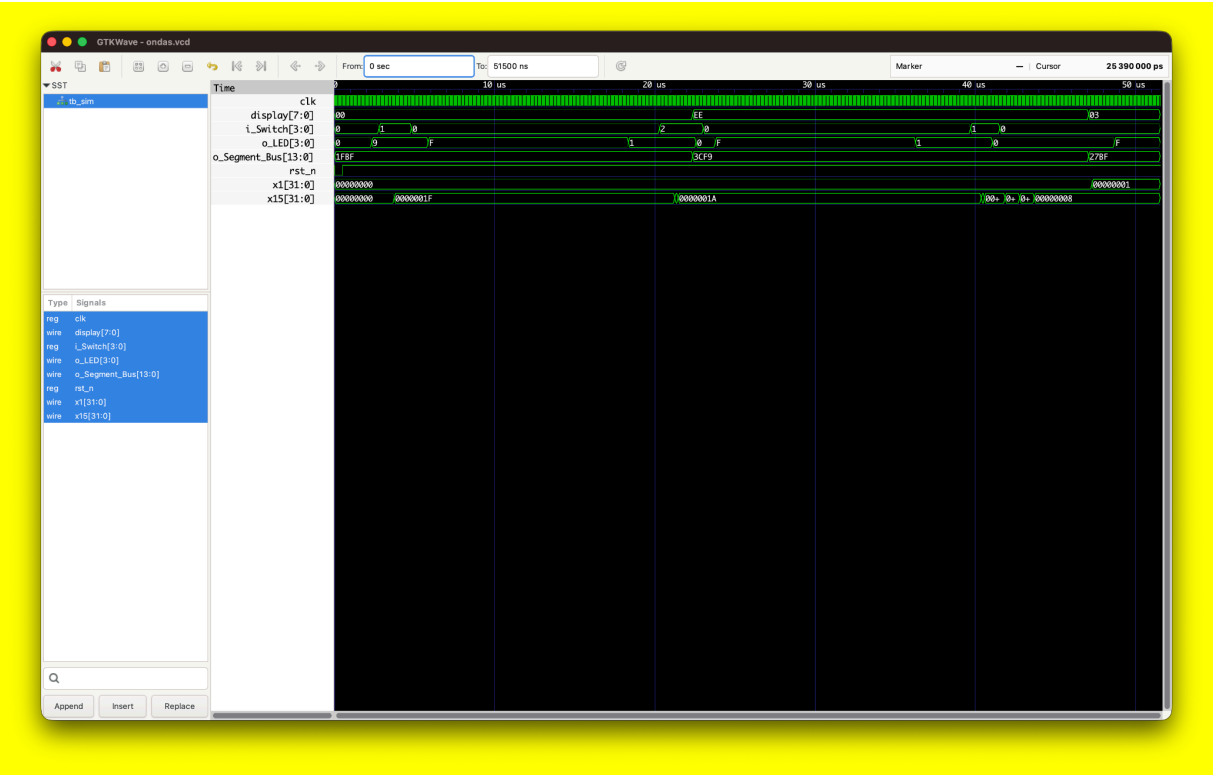


Figura 8.3: Simulación post-síntesis (Gate-Level) de la ronda correcta, validando que el tiempo medido en el registro x15 es idéntico al modelo RTL.

9. RESULTADOS Y CONCLUSIÓN

El desarrollo de este proyecto permitió abordar el desafío de llevar el sistema Pochoco desde su implementación original en FPGA hacia una implementación ASIC, considerando las adaptaciones necesarias tanto en el diseño como en su proceso de síntesis y verificación física. Este proceso requirió analizar las características propias de la tecnología de destino, tomar decisiones de implementación y evaluar distintas alternativas para conseguir un diseño funcional y eficiente.

En particular, el trabajo de adaptación del RTL permitió identificar y reemplazar elementos dependientes de la FPGA, además de estudiar dos estrategias para implementar la memoria del programa. La comparación entre ambas alternativas mostró que la solución basada en ROM combinacional y RAM de datos redujo aproximadamente un 78,6 % el área de celdas estándar respecto de la implementación mediante flip-flops, manteniendo resultados satisfactorios de verificación física. Estos resultados representan una parte de los avances obtenidos durante el proyecto y sirven como punto de partida para evaluar, en conjunto, el cumplimiento de los objetivos planteados.

Respecto de la síntesis, el resultado muestra que en un ASIC el costo del diseño está dominado por el almacenamiento: los flip-flops representan el 46 % del área, y el register file, construido completamente con ellos, ocupa más de la mitad del diseño. El contador de ciclos agregado en el Proyecto 2 es una fracción pequeña (2,94 %). Conservar la jerarquía de tres módulos permitió rastrear estas áreas con un costo de solo 1,05 % de área adicional.

En timing, el diseño cierra con amplio margen en el período objetivo de 100 ns y alcanza un período mínimo de 60,5 ns (16,5 MHz). La ruta crítica no está en un solo bloque, sino en la cadena decode, ALU, alineación de la LSU y write-back que Espino resuelve en un mismo ciclo, y se observó que el flujo no reoptimiza el circuito al reducir el período porque su reparación de timing solo considera el corner típico.

Por otro lado, la exploración del espacio de diseño (*Floorplan*) demostró que es posible compactar el núcleo ASIC hasta una utilización del 70%, aunque a costa de un aumento significativo en la longitud total de ruteo (*wirelength*) e incrementando los riesgos de congestión.

Por ello, la elección del 40% de utilización para la corrida final garantizó un enrutamiento holgado y un cierre exitoso de las reglas de diseño geométrico y eléctrico (DRC y LVS limpios).

Finalmente, las validaciones de *signoff* y la simulación funcional a nivel de compuertas (*Gate-Level*) verificaron que la inserción de diodos de protección fue efectiva contra violaciones de antena, y que el netlist final preserva intacta la lógica ciclo a ciclo del RTL original, concluyendo exitosamente la migración del SoC de FPGA a ASIC.

10. ANEXO

1.1. Código: rtl/pochoco_cycle_counter.v

```
module pochoco_cycle_counter (
  input  wire        clk_i,
  input  wire        rst_ni,
  output reg  [31:0] count_o
);
  always @(posedge clk_i or negedge rst_ni) begin
    if (!rst_ni) count_o <= 32'd0;
    else         count_o <= count_o + 32'd1;
  end
endmodule
```

Instancia en *rtl/pochoco_periph.v*, en lugar del registro del Proyecto 2:

```
wire [31:0] cycle_counter_q;
pochoco_cycle_counter u_cycle_counter (
  .clk_i  (clk_i),
  .rst_ni (rst_ni),
  .count_o(cycle_counter_q)
);
```

1.2. Código: asic/config_romram.json

```
{
  "DESIGN_NAME": "asic_top",
  "VERILOG_FILES": ["dir::../rtl/*.v", "dir::../rtl/espino_core/*.v"],
  "CLOCK_PORT": "i_Clk",
  "CLOCK_PERIOD": 100,
  "FP_CORE_UTIL": 40,
  "PDK": "gf180mcuD",
  "SYNTH_PARAMETERS": ["MemMode=1"],
  "TIMING_VIOLATION_CORNERS": ["*"],
  "SYNTH_KEEP_HIERARCHY_MODULES": ["espino_register_file", "espino_alu",
"pochoco_cycle_counter"],
  "GRT_ANTENNA_REPAIR_ITERS": 10,
  "GRT_ANTENNA_REPAIR_MARGIN": 20
}
```

1.3. Comandos para reproducir las corridas

Desde la raíz del repositorio y dentro del nix-shell de LibreLane:

```
# Regenerar game.hex, game_rom.v y game_mem_ff.v
make asm
# Corrida final (T = 100 ns)
make asic TAG=romram_final
# Un punto del barrido
make asic TAG=romram_T60.5 ARGS="-c CLOCK_PERIOD=60.5"
# Abrir el diseño en OpenROAD GUI
make openroad TAG=romram_T65
```